

FULL STACK DEVELOPMENT

FASE 4 | AULA 05 -

GERANDO FRAMEWORK E MICROFRONTENDS EM MOBILE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

EMANDA

O QUE VEM POR AÍ?

Nesta aula, exploraremos o conceito de frameworks e microfrontends em aplicações móveis. Discutiremos a importância da modularidade e reutilização de código, como criar frameworks personalizados e a implementação de microfrontends para melhorar a escalabilidade e a manutenção de aplicações móveis.

No hands on, implementaremos um framework básico e integraremos microfrontends em uma aplicação React Native, garantindo que diferentes partes da aplicação possam ser desenvolvidas e mantidas de forma independente.

HANDS ON

Começaremos criando um framework básico que encapsula funcionalidades comuns reutilizáveis em uma aplicação React Native. Configuraremos o projeto, definiremos e exportaremos componentes, utilitários e serviços. Em seguida, integraremos este framework em uma aplicação React Native, mostrando como utilizá-lo para construir diferentes partes da aplicação.

Passaremos, então, para a implementação de microfrontends, dividindo a aplicação em módulos independentes que podem ser desenvolvidos, testados e implantados separadamente.

Utilizaremos técnicas como lazy loading e microfrontend containers para integrar esses módulos de forma eficiente. Discutiremos a importância de usar frameworks e microfrontends para melhorar escalabilidade, modularidade e facilidade de manutenção das aplicações. Finalizaremos testando a aplicação para garantir que todos os módulos funcionem corretamente de forma integrada.

SAIBA MAIS

Um framework é uma coleção de bibliotecas, ferramentas e convenções que facilita o desenvolvimento de aplicações ao fornecer funcionalidades comuns e reutilizáveis. Em React Native, um framework pode incluir componentes UI (Interface de Usuário), serviços de API, utilitários e outras ferramentas que podem ser utilizados em diferentes partes da aplicação. Frameworks oferecem uma estrutura padronizada, permitindo que as pessoas desenvolvedoras se concentrem mais na lógica de negócios e menos na infraestrutura subjacente.

Benefícios dos frameworks

Frameworks aumentam a produtividade dos(as) desenvolvedores(as), fornecendo soluções prontas para problemas comuns e reduzindo a necessidade de escrever código do zero. Eles também garantem consistência no desenvolvimento de aplicações, pois seguem padrões e convenções estabelecidas.

Além disso, componentes e utilitários reutilizáveis permitem a economia de tempo e esforço, além de reduzir a duplicação de código, e facilitam a manutenção, pois manter e atualizar o código se torna mais fácil devido à estrutura organizada e modular fornecida por frameworks.

O que são microfrontends?

Microfrontends é uma abordagem arquitetural em que diferentes partes de uma aplicação são desenvolvidas, testadas e implantadas de forma independente. Cada microfrontend é um módulo autônomo que representa uma parte específica da aplicação, como uma funcionalidade ou um fluxo de usuário. Essa abordagem permite que diferentes equipes trabalhem em diferentes partes da aplicação sem interferir umas nas outras, promovendo uma maior modularidade e escalabilidade.

Por que usar Frameworks e Microfrontends?

Modularidade e reutilização

Frameworks permitem a reutilização de código, reduzindo a duplicação e facilitando a manutenção. Eles fornecem componentes e utilitários que podem ser usados em diversas partes da aplicação, promovendo a consistência e a eficiência.

Microfrontends dividem a aplicação em módulos independentes, permitindo que diferentes equipes trabalhem em partes distintas da aplicação de forma isolada. Isso melhora a organização e a modularidade do código.

Escalabilidade

A abordagem de microfrontends permite que uma aplicação cresça de maneira organizada e escalável, facilitando a adição de novos módulos sem impactar a estrutura existente. Cada microfrontend é autônomo e gerenciado de forma independente, o que significa que novos recursos podem ser desenvolvidos, testados e implantados sem afetar outras partes do sistema.

Essa modularidade não apenas simplifica a expansão da aplicação, mas também permite uma melhor distribuição de trabalho entre diferentes equipes, garantindo que o crescimento da aplicação seja sustentável e gerenciável.

Manutenção e atualizações

Manter e atualizar uma aplicação grande e complexa pode ser desafiador. A divisão da aplicação em microfrontends torna essa tarefa mais fácil e segura. Atualizações podem ser feitas em partes específicas da aplicação sem o risco de quebrar outras funcionalidades. Isso é possível porque cada microfrontend possui suas próprias dependências e ciclo de vida, permitindo que melhorias e correções sejam aplicadas de forma isolada. Essa independência reduz a complexidade das atualizações e facilita a introdução de novas tecnologias ou a refatoração de código antigo sem impactos significativos no restante da aplicação.

Desenvolvimento independente

Uma das maiores vantagens dos microfrontends é a capacidade de permitir que diferentes equipes trabalhem de forma independente em partes distintas da aplicação. Cada uma pode focar em um microfrontend específico, utilizando a tecnologia que melhor se adapta às suas necessidades. Essa independência promove a especialização e aumenta a produtividade, pois as equipes não precisam esperar

umas pelas outras para avançar com suas tarefas. Além disso, a colaboração entre equipes é facilitada, pois os pontos de integração são bem definidos e gerenciáveis.

Quando usar frameworks e microfrontends?

Frameworks e microfrontends são ideais para aplicações grandes e complexas que exigem modularidade, escalabilidade e manutenção contínua, tais como:

- Aplicações de comércio eletrônico: com múltiplas funcionalidades, como carrinho de compras, checkout e catálogo de produtos.
- Aplicações empresariais: com diferentes módulos para gerenciamento de clientes, produtos e vendas.
- Plataformas de mídia social: com diversas funcionalidades, como feed de notícias, mensagens e perfis de usuários.

A arquitetura de microfrontends tem ganhado popularidade devido à sua capacidade de modularizar aplicações complexas, permitindo que diferentes equipes trabalhem de forma independente em partes distintas da aplicação. Essa abordagem é particularmente eficaz em aplicações móveis desenvolvidas com React Native. Aqui estão alguns exemplos de empresas que utilizam microfrontends com React Native no mercado:

IKEA - Setor: varejo

A IKEA utiliza microfrontends para modularizar sua plataforma online, permitindo que diferentes equipes desenvolvam e implantem novas funcionalidades de maneira independente e ágil. Isso ajuda a garantir que a experiência do usuário seja consistente e que as atualizações sejam rápidas e eficientes.

Spotify - Setor: música e entretenimento

O Spotify adota microfrontends para criar uma plataforma modular e flexível. Essa arquitetura permite a entrega rápida de novas funcionalidades e uma maior agilidade no desenvolvimento, facilitando a gestão de diferentes componentes da aplicação de forma independente.

Zalando - Setor: moda e varejo

A Zalando implementa microfrontends para dividir sua aplicação em módulos menores e mais gerenciáveis. Isso facilita a escalabilidade e a manutenção da plataforma, além de permitir que diferentes equipes trabalhem em paralelo em várias funcionalidades.

Theodo - Setor: consultoria e desenvolvimento de software

A empresa Theodo utiliza microfrontends em projetos de clientes para escalar aplicações móveis e web. Utilizando React Native, eles conseguem implementar módulos independentes que podem ser desenvolvidos, testados e implantados separadamente, garantindo uma maior eficiência e flexibilidade no desenvolvimento.

Ionic (Portals) - Setor: desenvolvimento de software e ferramentas

A Ionic oferece o produto "Portals" para React Native, que permite a criação de experiências web modulares dentro de aplicativos móveis. Essa solução facilita a reutilização de código e a integração de funcionalidades desenvolvidas para web diretamente em aplicativos React Native, promovendo uma entrega rápida e eficiente de novas funcionalidades.

Como implementar - Criando um framework básico

Passo 1: configuração inicial do projeto

1. Configure um novo projeto React Native utilizando o CLI do React Native.
2. Crie um diretório para o framework em que os componentes reutilizáveis, utilitários e serviços serão definidos.

Passo 2: definindo componentes e utilitários

3. Crie componentes reutilizáveis, como botões, inputs e modais.
4. Implemente utilitários para manipulação de dados, formatação e outras funcionalidades comuns.

Passo 3: exportando o framework

5. Exporte os componentes e utilitários definidos para que possam ser utilizados em diferentes partes da aplicação.

Passo 4: integrando o framework em uma aplicação - Importando o framework

6. Importe os componentes e utilitários do framework na aplicação principal.
7. Utilize os componentes do framework para construir diferentes partes da aplicação, garantindo consistência e reutilização de código.

Passo 5: implementando microfrontends - Dividindo a aplicação em módulos

8. Identifique as diferentes partes da aplicação que podem ser desenvolvidas como microfrontends, como funcionalidades ou fluxos de usuário específicos.

Passo 6: criando microfrontends

9. Crie módulos independentes para cada microfrontend, com seus próprios componentes, serviços e estado.

Passo 7: integrando microfrontends na aplicação

10. Utilize técnicas como lazy loading para carregar os microfrontends sob demanda.
11. Implemente um container de microfrontends para gerenciar a integração dos diferentes módulos na aplicação principal.

Estrutura do projeto:

```
|-- framework/  
| |-- components/  
| | |-- Button.js  
| | |-- Input.js  
| |-- utils/  
| | |-- formatDate.js  
| |-- index.js  
|-- src/  
| |-- modules/  
| | |-- Cart/  
| | | |-- CartScreen.js  
| | |-- Checkout/  
| | | |-- CheckoutScreen.js  
| |-- App.js
```

```
|-- package.json
```

Componente button do framework:

```
import { TouchableOpacity, Text, StyleSheet } from 'react-native';

const Button = ({ title, onPress }) => {
  return (
    <TouchableOpacity style={styles.button} onPress={onPress}>
      <Text style={styles.text}>{title}</Text>
    </TouchableOpacity>
  );
};

const styles = StyleSheet.create({
  button: {
    backgroundColor: '#007bff',
    padding: 10,
    borderRadius: 5,
    alignItems: 'center',
  },
  text: {
    color: '#fff',
    fontSize: 16,
  },
});

export default Button;
```

Utilitário formatDate do framework:

```
export const formatDate = (date) => {
  const options = { year: 'numeric', month: 'long', day: 'numeric' };
  return new Date(date).toLocaleDateString(undefined, options);
};
```

Exportando o framework:

```
import Button from './components/Button';
import Input from './components/Input';
import { formatDate } from './utils/formatDate';
```

```
export {  
  Button,  
  Input,  
  formatDate,  
};
```

Integrando o framework na aplicação principal:

```
import { View, Text, StyleSheet } from 'react-native';  
import { Button } from '../../framework';  
  
const CartScreen = () => {  
  return (  
    <View style={styles.container}>  
      <Text>Cart Screen</Text>  
      <Button title="Checkout" onPress={() => console.log('Checkout pressed')} />  
    </View>  
  );  
};  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    justifyContent: 'center',  
    alignItems: 'center',  
  },  
});  
  
export default CartScreen;
```

Implementando e integrando microfrontends:

```
import { View, Text, StyleSheet } from 'react-native';  
import { Button } from '../../framework';  
  
const CheckoutScreen = () => {  
  return (  
    <View style={styles.container}>
```

```
<Text>Checkout Screen</Text>
  <Button title="Complete Purchase" onPress={() => console.log('Purchase
completed')} />
</View>
);
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
});

export default CheckoutScreen;
```

App.js:

```
import React, { lazy, Suspense } from 'react';
import { SafeAreaView, StyleSheet } from 'react-native';

const CartScreen = lazy(() => import('./modules/Cart/CartScreen'));
const CheckoutScreen = lazy(() => import('./modules/Checkout/CheckoutScreen'));

const App = () => {
  return (
    <SafeAreaView style={styles.container}>
      <Suspense fallback={<Text>Loading...</Text>}>
        <CartScreen />
        <CheckoutScreen />
      </Suspense>
    </SafeAreaView>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
  },
});
```

```
});  
  
export default App;
```

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre a importância e a implementação de frameworks e microfrontends em aplicações móveis. Discutimos como frameworks podem encapsular funcionalidades comuns reutilizáveis, enquanto microfrontends permitem modularizar a aplicação em partes independentes. Implementamos um framework básico, integrando componentes reutilizáveis na aplicação principal.

Em seguida, dividimos a aplicação em microfrontends, garantindo que cada módulo possa ser desenvolvido, testado e implantado separadamente. Abordamos técnicas como lazy loading para otimizar a performance e garantir uma experiência de usuário suave e responsiva. Concluimos que frameworks e microfrontends são essenciais para melhorar a escalabilidade, modularidade e facilidade de manutenção das aplicações móveis.

REFERÊNCIAS

Documentação oficial. **React Native**. [s.d.]. Disponível em: <https://reactnative.dev/docs/getting-started>. Acesso em: 09 ago. 2024.

LINDER, A. **Mastering React Native**: Full Stack Mobile App Development with React Native. [s.l.]: Packt Publishing, 2020.

OHNS, E. **React Native for Mobile Development: Harness the power of React Native to build efficient and feature-rich mobile applications**. [s.l.]: Packt Publishing, 2020.

PALAVRAS-CHAVE

Palavras-chave: React native. Front-end. Micro-frontend.

EMSE

POS TECH